DLDCA Inlab: Cache Me If You Can

Week 8

October 30, 2026

Introduction

This lab is about three things that decide how fast a program runs on real hardware: caches, branches and SIMD. For each one you measure the problem first, then fix it.

We use the tools from Lab 7 (`perf`, timers, pinning a run to a core), but on C++ programs instead of MIPS ones. Every task gives you a program that already runs. You write one function in it and compare the numbers before and after.

- **Task 1**: time one memory read for growing array sizes, and find your cache sizes from the plot.
- **Task 2**: speed up a matrix multiplication by reordering its loops.
- **Task 3**: remove a badly predicted branch and check the effect with `perf`.
- **Task 4**: do the same multiplication four floats at a time with SSE.

The tasks are not graded. The last hour is a graded lab exam.

Necessary tools

We use `g++` to compile, `perf` to count events, `matplotlib` to plot Task 1, and `lscpu` to read the cache sizes. Check that they are installed:

```
$ g++ --version
g++ ([linux-flavour] [version]) [version]
$ perf --version
perf version [version]
$ source ~/venv/bin/activate
(venv) $ python3 -c "import matplotlib; print(matplotlib.__version__)"
[version]
```

If `perf` is missing or fails with a permission error, follow the instructions from last week's lab, or use a Software Lab machine:

```
ssh <cse-username>@sl<N>-<MM>.cse.iitb.ac.in
```

Every task is built and run the same way. The program is compiled with `g++ -O0` and runs pinned to one core, so that its timings are not mixed between the fast and slow cores of a laptop:

```
$ make [task]                    # build [task] and run it on core 1
$ make [task] args="512"         # pass arguments to the program
$ make [task] CPU=2              # run on core 2 instead
$ make [task].out                # only build (used before perf)
```

The `perf` commands are written out in the tasks. They run the binary that `make` built.

Reference values. Each task ends with values measured on our machine. Yours might differ so compare the pattern not the numbers themselves.

Structure of submission/templates

Edit only the files marked `(TODO)`, and write your code only between the `YOUR CODE BELOW` and `YOUR CODE ABOVE` lines.

Submission format:

```
your-roll-no/
|- labexam/
    |- poly.cpp (TODO)
|- task1/
    |- latency.cpp (TODO)
    |- plot.py
|- task2/
    |- matmul.cpp (TODO)
|- task3/
    |- branchy.cpp (TODO)
|- task4/
    |- matmul_sse.cpp (TODO)
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz` in lowercase. The command given below can be used to do the same

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Tasks

Lab Exam: Poly Want a Speedup

`poly.cpp` applies a polynomial to every element of a 2048×2048 grid of floats:

$$out[i][j] = c_0 + c_1 x + c_2 x^2 + \cdots + c_7 x^7, \qquad x = in[i][j]$$

The polynomial is evaluated with Horner's rule, so each element takes 7 multiplications and 7 additions. `poly_slow` is the given implementation. Write `poly_fast`, which computes the same grid as fast as you can. Use what you learned in Tasks 2 and 4.

What you are given

`poly.cpp` contains everything except the body of `poly_fast`:

- the arrays `in`, `out_ref` and `out`, and the coefficients `coef[0..7]`;
- `poly_slow`, which writes `out_ref`;
- an empty `poly_fast`, which must write `out`. Until you fill it in, every element of `out` is wrong;
- `main`, which times both functions (best of 5 runs each) and compares `out` with `out_ref`.

Edit `poly_fast` only, between the `YOUR CODE BELOW` and `YOUR CODE ABOVE` lines. You may add helper code there. Code outside these lines must stay as it is. You may use `<xmmintrin.h>`; do not compile with other flags.

Grading

The program prints the speedup `poly_slow / poly_fast` and two lines, one per level. Both levels need a correct output: every element within 10^{-5} of `out_ref`.

Level	Needs	Speedup
1	correct output	at least 1.8×
2	correct output, and SSE instructions in `poly_fast`	at least 2.6×

Level 1 earns part of the marks and level 2 earns all of them.

Building and testing

All commands run from the `08_inlab/` directory:

```
$ make labexam            # build poly.cpp and run it once
slow:   ... ms
fast:   ... ms
speedup: ...x
wrong elements: 0
level 1 (>= 1.8x): ...
level 2 (>= 2.6x): ...
$ make labexam-check      # same, plus two more checks
PASS -- code outside the markers is unchanged
PASS -- poly_fast uses SSE (... mulps)
...
```

`make labexam-check` also checks that you did not edit the code outside the markers, and that `poly_fast` contains `mulps`. Level 2 counts only if that line says PASS. Timings vary between runs, so run it two or three times before you submit.

Reference values (the given `poly_fast`, loop order fixed only, and the full solution):

	given	loop order only	loop order + SSE
speedup	1.0×	2.2×	3.1×

Task 1: The Hills

Reading from L1 is fast, and reading from L2, L3 and DRAM is slower, in that order. You will time one read for arrays from 4 KB to 64 MB. The time jumps whenever the array no longer fits in a cache level, so the plot shows the cache sizes.

Step 1: Your cache sizes. Run `lscpu -C`. The `ONE-SIZE` column gives the size of one `L1d`, `L2` and `L3` cache.

You can also read the sizes of one core from `/sys`:

```
$ cat /sys/devices/system/cpu/cpu1/cache/index*/size
32K
32K
1024K
16384K
```

The four lines are L1d, L1i, L2 and L3.

Intel 12th gen and later: these CPUs have two kinds of cores, P-cores and E-cores, with different L1 and L2 sizes. `lscpu` mixes the two. Use the `/sys` command, with `cpu1`, the core `make` runs on.

Step 2: The program. A cache moves memory in 64-byte *cache lines*, which is 16 `int`s. The program reads one `int` per line (`STEP = 16`). Two things keep the measurement honest:

- The lines are visited in a **random order**. In order, the processor guesses the pattern and fetches ahead, and every read looks fast.
- Each read **depends on the previous one**. Otherwise the processor starts several reads at once and their waits overlap.

Both come from a linked list stored in the array: `next[i]` is the index of the next line to visit. The given setup builds it:

```
// order = 0, 1, 2, ..., lines-1, then shuffled
std::vector<long> order(lines);
std::iota(order.begin(), order.end(), 0);
std::shuffle(order.begin(), order.end(), std::mt19937(42));

// line order[k] points to line order[k+1]
std::vector<int> next(n);
for (long k = 0; k < lines; k++)
    next[order[k] * STEP] = order[(k + 1) % lines] * STEP;
```

With 4 lines and `order = [2, 0, 3, 1]`, this sets `next[32] = 0`, `next[0] = 48`, `next[48] = 16` and `next[16] = 32`. Starting at `i = 0`, repeating `i = next[i]` visits $0 \to 48 \to 16 \to 32 \to 0 \to \ldots$

Task. For each size, do the following:

1. **Warm-up:** do `i = next[i]` `lines` times, starting at `i = 0`. Not timed.
2. **Measure:** do `i = next[i]` `READS` times and time it with `std::chrono::steady_clock`.
3. **Print** in exactly this format (`plot.py` reads it):

```
std::printf("%ld %.2f\n", kb, ns / READS);
```

Fill in the marked part of `latency.cpp`.

Think: an array of 64 KB has how many cache lines? Is that more or fewer than the L1 of your machine can hold?

Run. Since we are creating plots, we need to have matplotlib available, so we need to activate the virtual environment present in `~/venv`.

```
$ source ~/venv/bin/activate
(venv) $ make task1
taskset -c 1 ./task1.out > task1/data.txt
saved .../latency.png
```

The output goes to `task1/data.txt`, and `plot.py` draws it on log axes in `task1/latency.png`.

Reference values. Time per read in ns, rounded. The L3 part is noisy.

KB	4	32	64	512	1024	2048	4096	16384	65536
ns	2	2	4	4.5	7	14	16	80–100	100

This CPU has 48 KB of L1d and 1.25 MB of L2 per P-core, and 24 MB of L3 shared by all cores.

Questions.

1. Where does the time jump? Label each flat part of your plot as L1, L2, L3 or DRAM.
2. Estimate each cache size from the jumps and compare with Step 1.
3. How many times slower is DRAM than L1?
4. Why does the time rise gradually near a cache size and not in one step?

Task 2: Out of Time

Code that reads memory in the wrong order misses the cache on almost every read, even with the same arithmetic. Here you only change the order of three loops.

Layout. C++ stores a 2D array row by row. In `float B[N][N]`, `B[k][0]`, `B[k][1]`, ... are neighbours in memory, and 16 of them share one 64-byte cache line. `B[0][j]`, `B[1][j]`, ... are a whole row apart. Walking along a row uses every float of each line fetched. Walking down a column fetches a new line for every element.

The given code. `matmul.cpp` computes $C = A \times B$ for $N \times N$ floats with the loop order i, j, k:

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        for (int k = 0; k < n; k++)
            C[i][j] += A[i][k] * B[k][j];
```

In the inner loop, `A[i][k]` walks along a row, but `B[k][j]` walks down a column.

Task. The loops can be in any order, and each `C[i][j]` still gets the same products. In `matmul.cpp (TODO)`, write

`void multiply_ikj(int n)`

with the loops in the order i, k, j. `A[i][k]` does not change inside the j loop, so read it into a local variable first.

Fill in `multiply_ikj` in `matmul.cpp`.

Think: in the new inner loop, which of `A`, `B` and `C` still change with j, and in which direction do they walk?

Run. The first argument is N (default 512). A second argument (`ijk` or `ikj`) runs just that version; without it both run and the checksums are compared.

```
$ make task2 args="512"
ijk: ... ms   checksum=...
ikj: ... ms   checksum=...
checksums match
$ taskset -c 1 perf stat -e task-clock,instructions,cycles,\
    L1-dcache-loads,L1-dcache-load-misses ./task2.out 512 ijk
$ taskset -c 1 perf stat -e task-clock,instructions,cycles,\
    L1-dcache-loads,L1-dcache-load-misses ./task2.out 512 ikj
```

Reference values ($N = 512$):

	ijk	ikj
time (ms)	480	260
instructions	5.7 G	4.5 G
L1-dcache-loads	1.9 G	1.6 G
L1-dcache-load-misses	59 M	1.3 M

N	128	256	512	1024
ijk (ms)	4.8	58	470	4200
ikj (ms)	4.3	34	254	1870

Questions.

1. How much faster is `ikj`? Both do N^3 multiply-adds, and both read memory on the order of N^3 times, but with different constant factors. Why? **Hint:** count cache lines, not floats; one line holds 16 floats. Assume $N \gg 16$ and that nothing stays in the cache from one run of the inner loop to the next. Approximate the number of cache lines each version fetches as $c \cdot N^3$, and find c for both.
2. Compare the L1 misses of the two versions and explain the difference.
3. Run $N = 128, 256, 512, 1024$. How does the speedup change with N? Relate it to your Task 1 plot.

Optional: Cachegrind. Cachegrind is not installed on the lab machines, so try it on your own Linux machine (`sudo apt install valgrind`). It runs the program on a simulated cache and counts the misses exactly, so the numbers are the same on every run. It is about 50 times slower, so use $N = 256$:

```
$ make task2.out
$ valgrind --tool=cachegrind --cache-sim=yes ./task2.out 256 ijk
$ valgrind --tool=cachegrind --cache-sim=yes ./task2.out 256 ikj
```

`valgrind` prints a summary at the end of each run. `D1 misses` counts the reads and writes that missed the L1 data cache. Compare it with N^3 and with your answer to question 1.

Each run also writes a file `cachegrind.out.<pid>`. For a detailed view, open it in KCachegrind (`sudo apt install kcachegrind`):

```
$ kcachegrind cachegrind.out.<pid>
```

- The **Flat Profile** on the left lists the functions, most expensive first. Click `multiply_ijk`.
- The drop-down in the toolbar picks the event that is counted. Switch it from instructions to `L1 Data Read Miss` (`D1mr`).
- The **Source Code** tab on the right shows each line of the function with its count. The line `C[i][j] += A[i][k] * B[k][j]` should get almost all the misses.

For more, ask a TA or read `man valgrind`.

Task 3: Wicked Games

A modern processor does not wait for a branch to be decided. It predicts the outcome and keeps executing on the predicted path. If the guess is right, the branch is almost free. If it is wrong, the work after the branch is thrown away, which costs many cycles. A branch that depends on random data cannot be predicted well.

The program. `branchy.cpp` fills an array `a` of $N = 32768$ `int`s with random values from 0 to 255 and runs this loop 2000 times:

```
for (int j = 0; j < N; j++) {
    if (a[j] >= T)          // T = 128
        s += a[j];
    else
        s -= a[j];
}
```

The first argument picks the version: `random` (the loop above), `sorted` (same loop, same values sorted first, sort not timed) or `branchless` (yours). Without an argument all three run and the sums are compared.

Think: the data is random, so `a[j] >= T` is true about half the time. What is the best a predictor can do on a coin flip?

Measure. Run the first two under `perf`. `branch-misses` counts the branches that were predicted wrong.

```
$ make task3.out
$ taskset -c 1 perf stat \
    -e task-clock,cycles,instructions,branches,branch-misses \
    ./task3.out random
$ taskset -c 1 perf stat \
    -e task-clock,cycles,instructions,branches,branch-misses \
    ./task3.out sorted
```

Task. A comparison like `(a[j] >= T)` is a number in C++: 1 if true, 0 if false. The processor computes it without a jump. Use that to turn the sign into -1 or $+1$ and multiply. In `branchy.cpp` `(TODO)`, write

```
long sum_branchless()
```

which returns the same value as `sum_branchy()`. **Inside the loops, do not use `if`, `else`, `?:`, `&&` or `||`.**

Fill in `sum_branchless` in `branchy.cpp`.

```
$ make task3
random    :    ... ms   sum=...
branchless:    ... ms   sum=...
sorted    :    ... ms   sum=...
sums match
$ taskset -c 1 perf stat \
    -e task-clock,cycles,instructions,branches,branch-misses \
    ./task3.out branchless
```

Reference values.

	random	sorted	branchless
time (ms)	305	126	130
cycles	1187 M	516 M	514 M
instructions	1156 M	1176 M	1580 M
branches	165 M	168 M	67 M
branch-misses	29.5 M	0.17 M	0.04 M

Questions.

1. `random` and `sorted` run almost the same number of instructions. Why is one so much slower?
2. What fraction of the 2000×32768 iterations mispredicted on random data?
3. Estimate the cost of one misprediction in cycles from the two runs.
4. Compute the IPC (`instructions / cycles`) of all three. Why is `branchless` as fast as `sorted` with more instructions?
5. How many branches per iteration does each version run? Which are left in `branchless`?

Optional. `perf record` samples an event while the program runs and saves where each sample happened in `perf.data`. `perf annotate` then prints the assembly of one function.

```
$ taskset -c 1 perf record -e branch-misses:pp ./task3.out random
$ perf annotate --stdio 'sum_branchy()'
```

Each line is `percent :  address instruction`, where the percent is the share of the samples that landed on that instruction. The `cmp` with `0x7f` is the test `a[j] >= T`. `:pp` asks the CPU to record the exact instruction. If `perf` says the event is not supported, drop `:pp`. For more, ask a TA or read `man perf-annotate`.

Which instruction gets the misses? Try again without `:pp` and see where the samples land.

Task 4: Four at a Time

SIMD instructions apply one operation to several values at once. An SSE register (`xmm`) is 128 bits wide and holds 4 `float`s, so one `mulps` does 4 multiplications. In C++ you use *intrinsics* from `<xmmintrin.h>`, one function per instruction. The type `__m128` holds 4 floats.

`_mm_set1_ps(x)`	all 4 lanes set to `x`
`_mm_loadu_ps(p)`	load 4 floats from address `p`
`_mm_mul_ps(a, b)`	lane-wise `a * b`
`_mm_add_ps(a, b)`	lane-wise `a + b`
`_mm_storeu_ps(p, v)`	store 4 lanes of `v` at address `p`

Task. The inner loop of the *ikj* multiply is `C[i][j] += a * B[k][j]` with `a = A[i][k]`. Neighbouring `j` are independent, so do 4 at a time: broadcast `a`, load 4 of `B[k][j]` and 4 of `C[i][j]`, multiply, add, store. In `matmul_sse.cpp (TODO)`, write

```
void multiply_sse(int n)
```

If `n` is not a multiple of 4, finish the last columns one at a time. Use only SSE intrinsics (see the table at the end).

Think: for `n = 510`, how many columns does the 4-wide loop cover per row, and how many are left over?

Fill in `multiply_sse` in `matmul_sse.cpp`.

Run.

```
$ make task4 args="512"
scalar:    ... ms   checksum=...
sse   :    ... ms   checksum=...
checksums match   speedup = ...x
$ make task4 args="510"
```

Reference values (speedup of `sse` over `scalar`):

n	64	512	1024
speedup	1.5	1.9	1.9

Questions.

1. What speedup do you get for `n = 512`? Why is it less than 4?
2. Try `n = 64` and `n = 1024`. Does it change much? What does that say about memory being the limit?
3. Run `n = 510`. Is the checksum correct? Which part of your code handles the last two columns?

Table 1: `perf stat` events used in this inlab

Event	Counts
`task-clock`	milliseconds the program ran on the CPU
`cycles`	clock cycles of the core
`instructions`	instructions that finished
`branches`	branch instructions executed
`branch-misses`	branches that were predicted wrong
`L1-dcache-loads`	reads from the L1 data cache
`L1-dcache-load-misses`	reads that were not in L1

IPC is `instructions / cycles`. On laptops with two kinds of cores, `perf` prints one row per kind (`cpu_core`, `cpu_atom`); read the row of the core you pinned to.

Table 2: Common `perf` commands

Command	Does
`perf list`	list the events this CPU can count
`perf stat -e EV1,EV2 ./prog`	count events over the whole run
`perf stat -r 5 -e EV ./prog`	run 5 times, print the mean and spread
`perf record -e EV ./prog`	sample where events happen, save to `perf.data`
`perf report --stdio`	functions from `perf.data`, most samples first
`perf annotate --stdio 'f()'`	assembly of `f` with samples per instruction

Put `taskset -c 1` in front of `perf stat` and `perf record` to pin the run.

Table 3: SSE intrinsics used in this inlab (`<xmmintrin.h>`)

Intrinsic	Instruction	Effect
`_mm_set1_ps(x)`	`shufps`	all 4 lanes = `x`
`_mm_loadu_ps(p)`	`movups`	load 4 floats from `p`
`_mm_storeu_ps(p, v)`	`movups`	store 4 floats to `p`
`_mm_add_ps(a, b)`	`addps`	4 additions
`_mm_mul_ps(a, b)`	`mulps`	4 multiplications
`_mm_sub_ps(a, b)`	`subps`	4 subtractions
`_mm_div_ps(a, b)`	`divps`	4 divisions
`_mm_min_ps(a, b)`	`minps`	lane-wise minimum
`_mm_max_ps(a, b)`	`maxps`	lane-wise maximum
`_mm_sqrt_ps(a)`	`sqrtps`	4 square roots
`_mm_setzero_ps()`	`xorps`	all 4 lanes = 0
`_mm_load_ps(p)`	`movaps`	like `loadu`, `p` must be 16-byte aligned
`_mm_store_ps(p, v)`	`movaps`	like `storeu`, `p` must be 16-byte aligned
`_mm_cvtss_f32(v)`	`movss`	the float in lane 0

The type `__m128` holds 4 floats. Do not compile with `-mavx` or `-march=native`: the Software Lab machines may not support the instructions they produce.

Table 4: Timing with `std::chrono` (`<chrono>`)

Code	Meaning
`auto t0 = std::chrono::steady_clock::now();`	current time
`auto d = t1 - t0;`	time between `t0` and `t1`
`std::chrono::duration<double, std::milli>(d).count()`	`d` in ms, as a `double`
`std::chrono::duration<double, std::nano>(d).count()`	`d` in ns, as a `double`
`std::chrono::duration<double>(d).count()`	`d` in s, as a `double`